Auto-deploy from GitHub

Push to main. CI passes. The server updates itself.

```
Ala Eh Stocks Monitoring System   |   Node + React + MySQL
Ubuntu 24.04   |   Nginx + PM2   |   GitHub Actions

GitHub Actions (CI) -> GitHub Actions (Deploy) -> SSH -> deploy.sh on the VPS
```

Every command is listed. Replace the UPPERCASE placeholders with your own values.

Contents

1 How it works

2 Placeholders used in this guide

3 Before you start (checklist)

4 Prepare the server

5 Let the server pull from GitHub (deploy key)

6 Let GitHub log in to the server (SSH key)

7 Add the secrets and variables on GitHub

8 The workflow files

9 Run it for the first time

10 Everyday use: push and it deploys

11 What a deploy changes, and what it never touches

12 Troubleshooting

13 Security notes

14 Command cheat sheet

1. How it works

You push your code to the `main` branch on GitHub. GitHub then runs two workflows, one after the other:

Order	Workflow	What it does
1	**CI** (`.github/workflows/ci.yml`)	Already in your repository. Lints, checks the migrations, type-checks, runs the tests, builds both parts, and runs the end-to-end test against a real MySQL. It must finish green.
2	**Deploy** (`.github/workflows/deploy.yml`)	New. Starts by itself only when CI succeeded for a push to `main`. Logs in to your server over SSH and runs the `deploy.sh` script your repository already has.

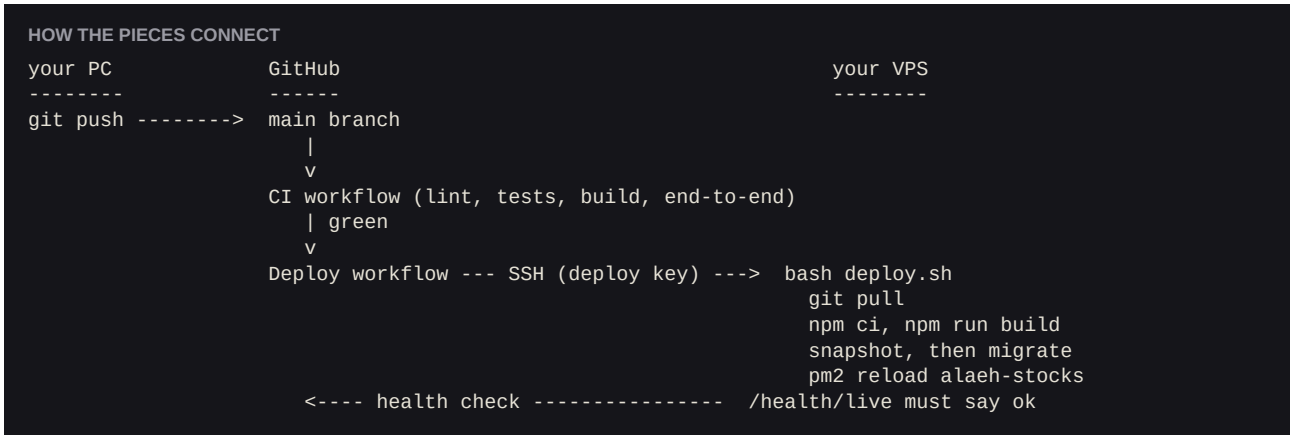
What the Deploy workflow does

- Checks that the exact commit has a successful CI run. If not, it refuses to deploy.
- Connects over SSH as the deploy user, using a key stored as a GitHub secret and a pinned server identity.
- Runs `bash /var/www/ala-eh/deploy/vps/deploy.sh` on the server.
- Checks the public site's health address from GitHub afterwards (optional, recommended).

What `deploy.sh` does on the server, in order

Step	Command	Why
1	`git pull --ff-only`	Fast-forward only. It never merges or overwrites local edits, and stops if it cannot.
2	`npm ci`	Installs the exact versions from package-lock.json.
3	`npm run build`	Builds the API (server/dist) and the client (client/dist, which Nginx serves).
4	`npm run db:migrate:deploy`	Checks the migrations, **takes a database snapshot first**, then migrates. If the snapshot fails, nothing is migrated.
5	`pm2 startOrReload ecosystem.config.cjs`	The API picks up the new code.
6	`curl /health/live`	Waits up to 30 seconds for the API to answer. If it does not, the run fails and says where to look.

Picture



NOTE Nothing here copies files from GitHub to the server. The server pulls its own code with git, so files git does not track (`server/.env`, backups, uploads) are never touched.

WARNING There is no approval step by default: every green push to `main` goes live. Section 13 shows how to require a manual approval click.

2. Placeholders used in this guide

Replace every placeholder below with your own value before running a command. The examples are the values from your VPS guide (docs/DEPLOY-VPS.md).

Placeholder	Meaning	Example
`YOUR_SERVER_IP`	Public IP address of the VPS	203.0.113.10
`DEPLOY_USER`	Linux user that owns the app folder and runs PM2 (never root)	deploy
`APP_DIR`	Folder of the app on the server	/var/www/ala-eh
`PM2_NAME`	The PM2 process that runs the API	alaeh-stocks
`API_PORT`	Port the API listens on (127.0.0.1 only)	4000
`YOUR_DOMAIN`	Public domain of the site	stocks.yourdomain.com
`GITHUB_USER` / `REPO_NAME`	GitHub account and repository	your-name / your-repo
`KEY_FILE`	Private key file you create for GitHub to use	ala_eh_deploy_key

NOTE Commands are marked with where to run them: **PowerShell on your PC**, **the server** (an SSH session) or **GitHub** (a web page).

3. Before you start (checklist)

All of these must already be true. If one is not, fix it first, because the deploy runs them with nobody watching.

Check	How to check	Where
The site already runs under PM2	`pm2 status` shows alaeh-stocks online	server
The app folder is a git clone of your repo	`cd /var/www/ala-eh && git remote -v`	server
Node 22 and npm are installed for that user	`node -v && npm -v && pm2 -v`	server
The settings file exists	`ls -l /var/www/ala-eh/server/.env`	server
The deploy script already works by hand	`bash /var/www/ala-eh/deploy/vps/deploy.sh` ends with 'Deployed OK'	server
The health address answers	`curl -s https://YOUR_DOMAIN/health/live` prints {"status":"ok"}	PC or server
CI exists and is green on main	`.github/workflows/ci.yml` is on main and passes	GitHub

CHECK The most important line is the fifth: if `deploy.sh` does not succeed when you run it yourself on the server, it will not succeed from GitHub either. Fix it by hand first (docs/DEPLOY-VPS.md, section 13).

4. Prepare the server

Do this once, logged in to the server as `DEPLOY_USER` (use `ssh DEPLOY_USER@YOUR_SERVER_IP`).

STEP 1 Check the folder owner and the tools

THE SERVER

```
ls -ld /var/www/ala-eh
node -v
npm -v
pm2 -v
git --version
```

`ls -ld` must show `DEPLOY_USER` as the owner. If it shows root, give the folder to the deploy user:

THE SERVER

```
sudo chown -R DEPLOY_USER:DEPLOY_USER /var/www/ala-eh
```

STEP 2 Make sure the working copy is clean

`git pull --ff-only` refuses to run if tracked files were edited on the server. Check:

THE SERVER

```
cd /var/www/ala-eh
git status
git log --oneline -3
```

`git status` should say *nothing to commit, working tree clean*. If it lists changed files, undo them with `git checkout -- FILE` (or move your edit into the repository and push it).

STEP 3 Check that PM2 and Node work over a plain SSH session

The workflow connects without a terminal, which loads fewer startup files than your normal login. From your PC:

POWERSHELL ON YOUR PC

```
ssh DEPLOY_USER@YOUR_SERVER_IP "node -v && npm -v && pm2 -v"
```

CHECK It must print three versions. If it says *command not found*, Node or PM2 was installed in a way that only loads for interactive logins (for example nvm). Install them system-wide as in docs/DEPLOY-VPS.md step 0 (NodeSource Node 22, then `sudo npm install -g pm2`).

STEP 4 You do not need sudo

Unlike a systemd setup, nothing here needs a passwordless sudo rule: PM2 runs under the deploy user, and `deploy.sh` refuses to run as root. Do not add the deploy user to the sudoers file for this.

NOTE `deploy.sh` also needs `server/.env` to exist and the database to be reachable with the `DATABASE_URL` in it. The snapshot and the migration use that same login.

5. Let the server pull from GitHub (deploy key)

The server runs `git pull` on its own, so it needs permission to read your repository. A GitHub **deploy key** gives read-only access to one repository and nothing else.

STEP 1 Create a key on the server

THE SERVER

```
ssh-keygen -t ed25519 -C "vps-github-pull" -N "" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

The second command prints one line starting with `ssh-ed25519`. Copy the whole line.

STEP 2 Add it to the repository on GitHub

- Open `https://github.com/GITHUB_USER/REPO_NAME/settings/keys`.
- Click **Add deploy key**.
- **Title:** `vps-pull` (any name).
- **Key:** paste the `ssh-ed25519 ...` line.
- Leave **Allow write access** unchecked. The server only needs to read.
- Click **Add key**.

STEP 3 Point the server's repo at GitHub over SSH and test

THE SERVER

```
ssh -T git@github.com
cd /var/www/ala-eh
git remote set-url origin git@github.com:GITHUB_USER/REPO_NAME.git
git pull
```

> **CHECK** `ssh -T` asks you to trust GitHub the first time: type **yes**. It then says *You've successfully authenticated.* `git pull` must run without asking for a username or password. If your repository is private, this deploy key is what makes that possible.

6. Let GitHub log in to the server (SSH key)

The Deploy workflow needs its own key pair to log in to the server. This is a **different** key from the one in section 5. Create it on **your PC**, not on the server.

STEP 1 Create the key pair on your PC (Windows PowerShell)

POWERSHELL ON YOUR PC

```
cd $HOME
ssh-keygen -t ed25519 -f "$HOME\KEY_FILE" -C "github-actions-deploy" -N '""'
dir $HOME\KEY_FILE*
```

This creates `KEY_FILE` (the **private** key: keep it secret) and `KEY_FILE.pub` (the public key). Run it in your user folder, not inside the project, so the private key can never be committed.

> **NOTE** In Windows PowerShell 5.1, `-N '""'` is needed for an empty passphrase. A plain empty pair of quotes is dropped and the command asks for a passphrase.

STEP 2 Install the public key on the server

`ssh-copy-id` does not exist on Windows, so send the key with this command. It asks for the user's password once:

POWERSHELL ON YOUR PC

```
Get-Content $HOME\KEY_FILE.pub | ssh DEPLOY_USER@YOUR_SERVER_IP "mkdir -p ~/.ssh && chmod 700
    ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

On Linux or macOS the same job is `ssh-copy-id -i ~/KEY_FILE.pub DEPLOY_USER@YOUR_SERVER_IP`.

STEP 3 Test the key

POWERSHELL ON YOUR PC

```
ssh -i $HOME\KEY_FILE DEPLOY_USER@YOUR_SERVER_IP
```

> **CHECK** It must log you in with no password prompt. Type `exit` to leave the server.

STEP 4 Get the server's host key line (DEPLOY_KNOWN_HOSTS)

This pins the server's identity so nobody can pose as it. Run this **on the server**:

THE SERVER

```
echo "YOUR_SERVER_IP $(cut -d' ' -f1,2 /etc/ssh/ssh_host_ed25519_key.pub)"
```

It prints one line: `YOUR_SERVER_IP ssh-ed25519 AAAA...`. That whole line is the value of the `DEPLOY_KNOWN_HOSTS` secret.

> **NOTE** Windows' built-in `ssh-keyscan` can fail with *unsupported KEX method*. Reading the key on the server, as above, avoids that and is more trustworthy. If SSH is not on port 22, start the line with `[YOUR_SERVER_IP]:PORT` instead.

STEP 5 Copy the private key text for GitHub

POWERSHELL ON YOUR PC

```
Get-Content $HOME\KEY_FILE -Raw | Set-Clipboard
```

This puts the whole private key on your clipboard, ready to paste into the `DEPLOY_SSH_KEY` secret. When the secret is saved, delete the files or keep them only in a password manager:

POWERSHELL ON YOUR PC (AFTER SAVING THE SECRET)

```
Remove-Item $HOME\KEY_FILE, $HOME\KEY_FILE.pub
```

7. Add the secrets and variables on GitHub

STEP 1 Create the `production` environment

- Open `https://github.com/GITHUB_USER/REPO_NAME/settings/environments`.
- Click **New environment**, name it exactly `production`, and click **Configure environment**.
- Optional: tick **Required reviewers** and add yourself, so each deploy waits for your approval click.

STEP 2 Add these environment secrets

In the `production` environment, under **Environment secrets**, click **Add environment secret** for each row:

Secret name	Value	Required
`DEPLOY_HOST`	YOUR_SERVER_IP (or the domain)	yes
`DEPLOY_USER`	DEPLOY_USER (for example deploy)	yes
`DEPLOY_SSH_KEY`	The full private key text, including the -----BEGIN OPENSSH PRIVATE KEY----- and -----END ...----- lines	yes
`DEPLOY_KNOWN_HOSTS`	The one line from section 6, step 4	yes (the workflow refuses to run without it)
`DEPLOY_PORT`	SSH port number	only if it is not 22

STEP 3 Add these variables

Under **Environment variables** in the same page:

Variable	What it is	Default in the workflow
`DEPLOY_PATH`	App folder on the server	/var/www/ala-eh
`DEPLOY_HEALTH_URL`	Public address GitHub checks after the deploy. It must answer `{"status":"ok"}`	empty (check skipped). Set it to https://YOUR_DOMAIN/health/live

> **NOTE** Your repository may already use a variable called `MONITOR_HEALTH_URL` for the 5-minute production monitor. That one is a repository variable and is separate. `DEPLOY_HEALTH_URL` is only for this workflow.

8. The workflow files

Both files live in `.github/workflows/` and are committed to the repository.

ci.yml (already in your repository, do not rename it)

The Deploy workflow starts when the workflow named **CI** finishes. Your `ci.yml` already starts with `name: CI` and runs on every push and pull request, with two jobs (`build-and-test` and `end-to-end`). **No change is needed.** A deploy starts only when both jobs are green.

> **NOTE** If CI is ever renamed, change `workflows: ["CI"]` in `deploy.yml` to match, or the deploy never starts.

deploy.yml (new file, full text)

> **NOTE** A ready-made copy of this file is saved next to this guide as `docs/deploy.yml.example`. Copy it to `.github/workflows/deploy.yml` instead of retyping it: copying text out of a PDF often loses the indentation, and YAML depends on it.

.GITHUB/WORKFLOWS/DEPLOY.YML (PART 1 OF 6)

```
name: Deploy

# Deploys to the VPS over SSH by running deploy/vps/deploy.sh there (git pull, npm ci,
# build, database snapshot + migrations, pm2 reload, wait for /health/live). The server
# runs under PM2 + Nginx, not Docker; see docs/DEPLOY-VPS.md section 13.
#
# Optional environment VARIABLES (Settings -> Environments -> production -> Variables):
#   DEPLOY_PATH        app folder on the server        (default /var/www/ala-eh)
#   DEPLOY_HEALTH_URL  public liveness address checked from GitHub after the deploy,
#                      e.g. https://stocks.yourdomain.com/health/live (skipped if empty)

on:
  # Automatic: every time CI finishes green for a push to main.
  workflow_run:
    workflows: ["CI"]
    types: [completed]
    branches: [main]
  # Manual: still available from the Actions tab (re-deploy, deploy a fix).
  workflow_dispatch:
```

.GITHUB/WORKFLOWS/DEPLOY.YML (PART 2 OF 6)

```
# One deploy at a time; a newer push waits for the running one instead of racing it,
# and an in-progress deploy is never cancelled half way.
concurrency:
  group: production-deploy
  cancel-in-progress: false

jobs:
  deploy:
    name: Deploy to production
    runs-on: ubuntu-latest
    environment: production
    # Auto-deploy only after a successful CI run of a push to main (never for a pull
    # request or a failed run). A manual run must be started on main.
    if: >-
      (github.event_name == 'workflow_dispatch' && github.ref == 'refs/heads/main') ||
      (github.event.workflow_run.conclusion == 'success' &&
       github.event.workflow_run.event == 'push')
    env:
      # The commit CI actually tested. For workflow_run, github.sha is just the newest
      # commit on main, which may not be the one that passed.
      DEPLOY_SHA: ${{ github.event.workflow_run.head_sha || github.sha }}
      DEPLOY_PATH: ${{ vars.DEPLOY_PATH || '/var/www/ala-eh' }}
      DEPLOY_HEALTH_URL: ${{ vars.DEPLOY_HEALTH_URL }}
    permissions:
```

.GITHUB/WORKFLOWS/DEPLOY.YML (PART 3 OF 6)

```
      contents: read
      actions: read

    steps:
      # Refuse to deploy a commit whose CI run has not finished green.
      - name: Require a successful CI run for this commit
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          result="$(gh run list --repo "${{ github.repository }}" --workflow ci.yml \
            --commit "$DEPLOY_SHA" --json conclusion \
            --jq '[.[] | select(.conclusion == "success")] | length')"
          if [ "${result:-0}" -lt 1 ]; then
            echo "No successful CI run found for $DEPLOY_SHA; refusing to deploy." >&2
            exit 1
          fi
```

.GITHUB/WORKFLOWS/DEPLOY.YML (PART 4 OF 6)

```
      - name: Set up SSH
        env:
          DEPLOY_SSH_KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          DEPLOY_KNOWN_HOSTS: ${{ secrets.DEPLOY_KNOWN_HOSTS }}
        run: |
          # The pinned host key is required: trusting the server on first contact would
          # let someone on the network capture the deploy session.
          if [ -z "$DEPLOY_KNOWN_HOSTS" ]; then
            echo "DEPLOY_KNOWN_HOSTS secret is not set; refusing to deploy." >&2
            exit 1
          fi
          mkdir -p ~/.ssh
          chmod 700 ~/.ssh
          printf '%s\n' "$DEPLOY_SSH_KEY" > ~/.ssh/deploy_key
          chmod 600 ~/.ssh/deploy_key
          printf '%s\n' "$DEPLOY_KNOWN_HOSTS" > ~/.ssh/known_hosts
```

.GITHUB/WORKFLOWS/DEPLOY.YML (PART 5 OF 6)

```
      # deploy.sh pulls, installs, builds, snapshots + migrates the database, restarts
      # PM2 and waits for the API. It stops at the first thing that fails.
      - name: Run deploy.sh on the server
        env:
          DEPLOY_HOST: ${{ secrets.DEPLOY_HOST }}
          DEPLOY_USER: ${{ secrets.DEPLOY_USER }}
          DEPLOY_PORT: ${{ secrets.DEPLOY_PORT }}
        run: |
          ssh -i ~/.ssh/deploy_key -o IdentitiesOnly=yes -p "${DEPLOY_PORT:-22}" \
            "$DEPLOY_USER@$DEPLOY_HOST" \
            "bash '$DEPLOY_PATH/deploy/vps/deploy.sh'"

      - name: Check the public site
        if: ${{ env.DEPLOY_HEALTH_URL != '' }}
        run: |
          for i in $(seq 1 15); do
            if curl -fsS --max-time 10 "$DEPLOY_HEALTH_URL" | grep -q '"status":"ok"'; then
              echo "public site is healthy"; exit 0
            fi
            sleep 4
          done
          echo "the public health address did not answer ok" >&2
          exit 1
```

.GITHUB/WORKFLOWS/DEPLOY.YML (PART 6 OF 6)

```
      - name: Clean up SSH key
        if: always()
        run: rm -f ~/.ssh/deploy_key
```

How the important parts work

Part	Purpose
`on: workflow_run (CI, completed, main)`	Starts the deploy automatically when CI has finished for `main`.
`on: workflow_dispatch`	Adds a **Run workflow** button so you can deploy or re-deploy by hand. It only runs from main.
`if: ... conclusion == 'success' && event == 'push'`	No deploy for a failed CI run or for a pull request.
`concurrency: production-deploy`	Only one deploy at a time. A second push waits for the first.
`DEPLOY_SHA`	The commit CI actually tested, not just the newest commit on `main`.
Require a successful CI run	Asks GitHub whether that commit has a green CI run. Refuses otherwise.
`environment: production`	Gives the job access to the secrets, and to the approval gate if you enabled it.
secrets passed as `env:`	The key and host line reach the shell as environment variables, never pasted into the script text.
`bash 'DEPLOY_PATH/deploy/vps/deploy.sh'`	Runs your existing script. All the real update logic stays in one place you can also run by hand.
`grep -q '"status":"ok"'`	The public check passes only if the health body says ok, so a redirect or an error page cannot fake success.

> **NOTE** The server updates to the newest `main`, which can be a commit or two ahead of the one CI just tested if you push twice quickly. The second push runs its own CI and deploy right after, so the server always ends on the last green commit.

9. Run it for the first time

STEP 1 Commit and push the workflow

POWERSHELL ON YOUR PC

```
cd "C:\Programming\work-repos\Monitoring System"
git add .github/workflows/deploy.yml
git commit -m "ci: deploy to the server after CI passes"
git push
```

WARNING This push, and every push to `main` after it, goes live once CI is green. Make sure everything already on `main` is ready, or turn on the approval gate (section 13) before pushing.

STEP 2 Watch it

- Open `https://github.com/GITHUB_USER/REPO_NAME/actions`.
- First the **CI** workflow runs. Wait for the green tick.
- Then the **Deploy** workflow starts by itself. Open it and click the **Run deploy.sh on the server** step to read the log.
- A successful run ends with the line `Deployed OK (...)`, then `public site is healthy`.

STEP 3 Or start it by hand without pushing anything

- Actions -> **Deploy** (left list) -> **Run workflow** -> choose branch **main** -> **Run workflow**.
- A manual run still needs a green CI run for that commit.

STEP 4 Verify on the server

THE SERVER

```
cd /var/www/ala-eh
git log --oneline -1
pm2 status
ls -lh /var/backups/alaeh-stocks/snapshots | tail -3
curl -s https://YOUR_DOMAIN/health/live
pm2 logs alaeh-stocks --lines 30 --nostream
```

CHECK `git log` shows the hash of your latest commit. `alaeh-stocks` is **online** with a recent restart time. The snapshots folder has a new file from this deploy. The health address prints `{"status":"ok"}`. The log has no errors.

Then open the site in the browser and press **Ctrl+Shift+R** to bypass the cache. The app is installable, so a service worker can keep the old screen until you refresh.

10. Everyday use: push and it deploys

POWERSHELL ON YOUR PC

```
git add .
git commit -m "your message"
git push
```

That is all. GitHub runs CI, then deploys. You do not log in to the server.

Rules of thumb

- **CI red:** nothing is deployed. Read the failing step in the Actions tab and fix it.
- **Deploy red:** the server may be half updated (for example the build passed but a migration did not). Read the log, fix the cause and click **Run workflow** again.
- **Every push restarts the API** for a few seconds. Group small changes into one push when you can.
- **Do not push while people are in the middle of a save, a publish, or a restore.** The restart interrupts the request in flight. Do it after the shift's entries are saved.
- **Do not push while a database restore is running** on the server (`scripts/restore.sh`).
- **Delete a file** with `git rm FILE`, commit and push. The server's `git pull` deletes it there too.
- **New settings:** if a push adds variables to `.env.example`, add them to the server's `server/.env` by hand and run `pm2 restart alaeh-stocks --update-env`. The deploy never edits `.env`.

Roll back to an older version

Quickest: revert the bad commit and push, so the normal pipeline deploys the fix.

POWERSHELL ON YOUR PC

```
git revert BAD_COMMIT_HASH
git push
```

To put the server on an older commit right now, without a new push, on the server:

THE SERVER

```
cd /var/www/ala-eh
git log --oneline -10
git reset --hard GOOD_COMMIT_HASH
npm ci && npm run build
pm2 restart alaeh-stocks --update-env
# later, when the fix is pushed, the next deploy fast-forwards the server again
```

> **WARNING** A rollback does **not** undo database migrations. If a migration has to be undone, restore the snapshot the deploy took (next block). Anything saved after that snapshot is lost, so use it as a last resort.

Restore the database from the pre-migration snapshot

THE SERVER

```
pm2 stop alaeh-stocks
ls -lh /var/backups/alaeh-stocks/snapshots
bash /var/www/ala-eh/scripts/restore.sh --file /var/backups/alaeh-stocks/snapshots/FILE --yes
pm2 start alaeh-stocks
```

`restore.sh` opens encrypted backups itself (it needs the same `BACKUP_ENCRYPTION_KEY` in `server/.env`), checks the file is a complete dump before touching anything, and takes a safety snapshot first. Only the newest 10 snapshots are kept.

11. What a deploy changes, and what it never touches

A deploy changes	A deploy never touches
The code: server/dist and client/dist are rebuilt. Installed packages (exactly as locked). The database **structure**, but only through new reviewed migrations, after a snapshot. The running API process (restarted).	Saved stock data: entries, counts, audits, openings and reports are not rewritten by a deploy. A code change that alters how *future* saves or publishes behave does not re-run on old records. `server/.env` and every secret in it. Backups, uploads and the Google Drive connection. Nginx and the certificate.

> **NOTE** If a release changes how a calculation works (for example how an audit sets the next day's opening), the new rule applies to what is saved **after** the deploy. Existing records change only when someone redoes them in the app, for example by reverting and re-publishing an audit. Plan that as a separate step after the deploy, and take a backup first.

Migrations are the one risky part

- CI runs `npm run check:migrations` and fails on data-risky statements that were not reviewed. Fix or acknowledge them in the repository before the push can deploy.
- `deploy.sh` takes a snapshot first and stops if the snapshot fails, so a failed snapshot means **nothing** was migrated.
- Keep schema changes backward compatible: the old code still runs for the few seconds between the migration and the restart.
- Copy the backups off the server regularly (docs/DEPLOY-VPS.md, section 11). A backup next to the database is lost with it.

The 5-minute production monitor

Your repository also has `production-monitor.yml`, which checks the live site every 5 minutes. A deploy restarts the API for a few seconds and the monitor retries before it reports a failure, so a normal deploy does not raise an alert. A deploy that stays down will.

12. Troubleshooting

What you see	Cause	Fix
`DEPLOY_KNOWN_HOSTS secret is not set; refusing to deploy.`	The secret is missing or empty	Add it in the production environment (section 6, step 4).
`No successful CI run found for ...; refusing to deploy.`	CI did not finish green for that commit	Fix CI, push again, or wait for CI to finish and run the deploy again.
Deploy never starts after CI	Workflow name is not CI, the branch is not main, or the run was a pull request	Check `workflows: ["CI"]` and `branches: [main]`. The file must be on main.
`Permission denied (publickey)` in the log	The public key is not in authorized_keys, or the wrong private key is in the secret	Redo section 6, steps 2 and 3. Paste the whole private key, including the BEGIN and END lines.
`Host key verification failed`	DEPLOY_KNOWN_HOSTS does not match the server	Recreate the line on the server (section 6, step 4). Check the IP and the port.
`choose_kex: unsupported KEX method` from ssh-keyscan	Old Windows OpenSSH	Read the host key on the server instead (section 6, step 4).
`ERROR: Do not run this as root.`	DEPLOY_USER is root	Use the normal user that owns the app folder and runs PM2.
`pm2: command not found` or `npm: command not found`	Installed for interactive logins only (nvm, a user prefix)	Install Node and PM2 system-wide (section 4, step 3).
`ERROR: server/.env is missing.`	Wrong DEPLOY_PATH, or the file was never created	Check the variable and docs/DEPLOY-VPS.md step 5.
`git pull` asks for a password, or fails with Permission denied	Deploy key missing, or the remote is an https URL	Section 5. Run `git remote -v`; it must show git@github.com:....
`fatal: Not possible to fast-forward`	The server has local commits or edited tracked files	On the server: `git status`. Undo the local edits, then re-run.
The build dies with `Killed` or runs out of memory	A small VPS cannot build the client	Add swap space (a 2 GB swap file), then re-run.
CI or the deploy stops at `check:migrations`	A migration has an unreviewed data-risky statement	Review it in the repository; the deploy never runs it until that passes.
The snapshot step fails	The database login or the disk is the problem. Nothing was migrated.	Check `DATABASE_URL` in server/.env and free space in /var/backups/alaeh-stocks.
`The API did not answer`	The API crashed on start (a missing or too-short setting is named in the log)	`pm2 logs alaeh-stocks --lines 50`. Fix server/.env, then `pm2 restart alaeh-stocks --update-env`.
deploy.sh passes but the public check fails	DEPLOY_HEALTH_URL is wrong, DNS or HTTPS is down, or Nginx lost its proxy	Open the address in a browser. Check `sudo nginx -t` and the certificate.
502 Bad Gateway after a deploy	The API is not running	`pm2 status`, then the logs.
Old look or old screens after a deploy	Browser or service-worker cache	Press Ctrl+Shift+R. If it persists, clear the site's data in the browser.
`WebSocket` errors in the browser console	Nginx lacks the /ws block	Use the config in `deploy/nginx/ala-eh.conf` (docs/DEPLOY-VPS.md section 8), then `sudo nginx -t` and reload.

Useful log commands

THE SERVER

```
pm2 logs alaeh-stocks --lines 100 --nostream
pm2 logs alaeh-stocks
sudo tail -n 50 /var/log/nginx/error.log
cd /var/www/ala-eh && git status && git log --oneline -5
```

13. Security notes

- **Use a dedicated key** for GitHub-to-server. Do not reuse your personal SSH key. If it leaks, remove just that line from `~/.ssh/authorized_keys`.
- **Never commit** the private key, `server/.env`, or backup files.
- **Do not deploy as root.** The deploy user needs only the app folder and PM2. The script refuses to run as root anyway.
- **Keep DEPLOY_KNOWN_HOSTS required.** Trusting a key on first contact would let someone on the network capture the session.
- **The deploy key is read-only.** Leave 'Allow write access' off.
- **Keep NODE_ENV=production** (PM2 sets it) and keep `JWT_SECRET` and `DATA_RESET_PASSCODE` private. The server refuses to start without them.
- **Rotate** the deploy keys if a laptop is lost or a teammate leaves: create new keys, replace the secret, delete the old public key from the server and from GitHub.
- **Protect main:** in the repository's branch protection, require the CI checks to pass and block force pushes, so nothing reaches production without a green run.

Require a manual approval before each deploy (optional)

- Settings -> Environments -> `production`.
- Tick **Required reviewers**, add yourself or teammates, and click **Save protection rules**.
- After that, a green CI run starts the Deploy workflow, which then waits. A reviewer clicks **Review deployments -> Approve** to let it run.

> **WARNING** Migrations run on every deploy and there is no automatic rollback for them. The snapshot in /var/backups/alaeh-stocks/snapshots is your safety net. Copy the backups off the server regularly.

14. Command cheat sheet

One-time setup, in order

THE SERVER

```
# --- the server ---
ls -ld /var/www/ala-eh && node -v && npm -v && pm2 -v
bash /var/www/ala-eh/deploy/vps/deploy.sh              # must end with "Deployed OK"
ssh-keygen -t ed25519 -C "vps-github-pull" -N "" -f ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub               # paste into GitHub -> Settings -> Deploy keys
ssh -T git@github.com
cd /var/www/ala-eh && git remote set-url origin git@github.com:GITHUB_USER/REPO_NAME.git
git pull
echo "YOUR_SERVER_IP $(cut -d' ' -f1,2 /etc/ssh/ssh_host_ed25519_key.pub)"   # DEPLOY_KNOWN_HOSTS
```

POWERSHELL ON YOUR PC

```
# --- your PC (PowerShell) ---
cd $HOME
ssh-keygen -t ed25519 -f "$HOME\KEY_FILE" -C "github-actions-deploy" -N '"'
Get-Content $HOME\KEY_FILE.pub | ssh DEPLOY_USER@YOUR_SERVER_IP "mkdir -p ~/.ssh && chmod 700
    ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
ssh -i $HOME\KEY_FILE DEPLOY_USER@YOUR_SERVER_IP
Get-Content $HOME\KEY_FILE -Raw | Set-Clipboard    # paste into the DEPLOY_SSH_KEY secret
```

GITHUB

```
# --- GitHub (web) ---
Settings -> Environments -> New environment: production
Secrets:   DEPLOY_HOST  DEPLOY_USER  DEPLOY_SSH_KEY  DEPLOY_KNOWN_HOSTS  (DEPLOY_PORT if not 22)
Variables: DEPLOY_PATH (if not /var/www/ala-eh)   DEPLOY_HEALTH_URL (recommended)
```

Every day

POWERSHELL ON YOUR PC

```
git add .
git commit -m "message"
git push
# GitHub: CI -> Deploy -> server updates itself
```

Checks after a deploy

THE SERVER

```
cd /var/www/ala-eh && git log --oneline -1
pm2 status
ls -lh /var/backups/alaeh-stocks/snapshots | tail -3
curl -s https://YOUR_DOMAIN/health/live
pm2 logs alaeh-stocks --lines 30 --nostream
```

Manual deploy on the server (same steps as the workflow)

THE SERVER

```
bash /var/www/ala-eh/deploy/vps/deploy.sh
```